

整数論 合同式 環

合同式の基礎

[解説動画はこちら](#)です。

1. 概要

本講座では**合同式**の基礎的な考え方について説明します。

合同式は、整数全体を m で割ったときの余り（**剰余**）に応じて分類するものです。競技プログラミングでは特に「答えを m で割った余りで出力せよ」という形式が頻出で、そのような問題では合同式を適切に扱えることが前提となります。

2. 前提知識

特にありません。

3. 合同式と剰余

3.1. 合同式

【定義 1】

m を正整数とする。整数 a, b に対して $a - b$ が m の倍数であるとき、つまり $a - b = cm$ となる整数 c が存在するとき、 a と b は m を法として**合同**（congruent）であるといい、

$$a \equiv b \pmod{m}$$

と書く．合同関係の現れる式を合同式という．

例えば次が成り立ちます．

- $1234 \equiv 234 \pmod{5}$.
- $1234 \equiv 4 \pmod{5}$.
- $123 \equiv 3 \pmod{10}$.
- $-123 \equiv 7 \pmod{10}$.
- $9999 \equiv -1 \pmod{10}$.
- $1 \equiv 0 \pmod{1}$.

$a \equiv b \pmod{m}$ かつ $b \equiv c \pmod{m}$ ならば, $a \equiv c \pmod{m}$ が成り立ちます．このことから, $a \equiv b \pmod{m}$ かつ $b \equiv c \pmod{m}$ であることを $a \equiv b \equiv c \pmod{m}$ などと繋げて表記することもよくあります．

文脈上, m を固定して議論することもよくあります．この場合には, $(\pmod{m})$ は省略されることもあります．

例えば 3 を法とすると, 次が成り立ちます．

$$\begin{aligned}\dots &\equiv -9 \equiv -6 \equiv -3 \equiv 0 \equiv 3 \equiv 6 \equiv \dots \\ \dots &\equiv -8 \equiv -5 \equiv -2 \equiv 1 \equiv 4 \equiv 7 \equiv \dots \\ \dots &\equiv -7 \equiv -4 \equiv -1 \equiv 2 \equiv 5 \equiv 8 \equiv \dots\end{aligned}$$

このように m を法とする合同によって, 整数全体は m 種類に分類されます．

3.2. 剰余

$\pmod$ という表記は, 合同式だけでなく剰余を表す際にも用いられます．

【定義 2】

m を正整数とする．整数 a に対して $a \equiv r \pmod{m}$ かつ $0 \leq r < m$ を満たす非負整数 r が唯一存在する (証明は省略)．この r を a の m による**剰余** (remainder) と

いい,

$$a \bmod m$$

と書く.

例えば次が成り立ちます.

- $1234 \bmod 5 = 4.$
- $123 \bmod 10 = 3.$
- $-123 \bmod 10 = 7.$
- $9999 \bmod 10 = 9.$
- $1 \bmod 1 = 0.$

また, $a \equiv (a \bmod m) \pmod{m}$ が成り立ちます.

$\lfloor x \rfloor$ を**床関数** (floor function), つまり x 以下の最大の整数とすると,

$$a \bmod m = a - m \cdot \left\lfloor \frac{a}{m} \right\rfloor$$

が成り立ちます. (床関数については [競技プログラミング基本用語集](#) 参照.)

$a \bmod m$ の値は, m を法として a と合同な数のうち最も単純な値を求めていると考えることもできます. $a \bmod m$ の値を求めることを, a を $\bmod m$ で**求める**ということもあります.

▶ 注意: プログラミング言語における剰余

4. 合同式の性質

【定理 3】

$a_1 \equiv a_2 \pmod{m}$, $b_1 \equiv b_2 \pmod{m}$ であるとき, 次が成り立つ.

$$a_1 + b_1 \equiv a_2 + b_2 \pmod{m}$$

$$a_1 - b_1 \equiv a_2 - b_2 \pmod{m}$$

$$a_1 b_1 \equiv a_2 b_2 \pmod{m}$$

いずれも次の式変形により、左辺と右辺の差が m の倍数の和として表されていることから証明されます。

$$\begin{aligned}(a_1 + b_1) - (a_2 + b_2) &= (a_1 - a_2) + (b_1 - b_2) \\(a_1 - b_1) - (a_2 - b_2) &= (a_1 - a_2) - (b_1 - b_2) \\a_1 b_1 - a_2 b_2 &= a_1(b_1 - b_2) + b_2(a_1 - a_2)\end{aligned}$$

■

これらの性質から、いくつかの数から加算・減算・乗算の反復によって得られる値を $\text{mod } m$ で計算するときには、

- 最初に与えられた値をそれと合同な数に置き換えてもよい。
- 加算・減算・乗算などを行うたびに、その計算結果をそれと合同な数に置き換えてもよい。

ことが分かります。例えば $123 \times 456 + 789$ を $\text{mod } 10$ で計算すると、

- $123 \times 456 \equiv 3 \times 6 \equiv 8 \pmod{10}$
- $789 \equiv 9 \pmod{10}$
- よって $123 \times 456 + 789 \equiv 8 + 9 \equiv 7 \pmod{10}$

のように計算できます。

4.1. 練習問題

競技プログラミング形式の問題ではなく、手計算の前提で練習問題を用意したので、挑戦してみてください。

【問題 4】

次を求めてください。

1. $(53 \times 23 \times 32 - 33 \times 44) \text{ mod } 7$
2. $(999 \times 789) \text{ mod } 1000$
3. $2^{123} \text{ mod } 24$

▶ 解説

【問題 5】

m, a_1, a_2, b_1, b_2 はいずれも正整数であるとして、 $a_1 \equiv a_2 \pmod{m}$, $b_1 \equiv b_2 \pmod{m}$ であるときに次の性質が常に必ず成り立つかを判定し、成り立たない場合には反例を挙げてください。

1. a_1 が b_1 の倍数, a_2 が b_2 の倍数であるとき $\frac{a_1}{b_1} \equiv \frac{a_2}{b_2} \pmod{m}$.
2. 非負整数 n に対して $a_1^n \equiv a_2^n \pmod{m}$.
3. $a_1^{b_1} \equiv a_2^{b_2} \pmod{m}$.
4. $a_1! \equiv a_2! \pmod{m}$.
5. $p(x) = \sum_{i=0}^n p_i x^i$ を整数係数多項式とすると、 $p(a_1) \equiv p(a_2) \pmod{m}$.

▶ 解説

5. 環 $\mathbb{Z}/m\mathbb{Z}$

$\text{mod } m$ での加算・減算・乗算によって集合 $\{0, 1, \dots, m-1\}$ に対して、演算を定めたもの（集合と演算の組）を、 $\mathbb{Z}/m\mathbb{Z}$ と書くことにします。

例えば $m = 4$ とするとき、加算・減算・乗算は次の演算表のようになります。

+	0	1	2	3
0	0	1	2	3
1	1	2	3	0
2	2	3	0	1
3	3	0	1	2

−	0	1	2	3
0	0	3	2	1
1	1	0	3	2
2	2	1	0	3
3	3	2	1	0

×	0	1	2	3
0	0	0	0	0
1	0	1	2	3
2	0	2	0	2
3	0	3	2	1

▶ 定義や記号に関する注意

数学ではこのように、集合と演算の組をひとつの数学的対象として扱うことがあり、特に集合に加算・減算・乗算を定めたものであって、いくつかの性質を満たすものは環（ring）と呼ばれます。（本講座では正確な定義までを理解する必要はありませんが、必要であれば 代数構造用語集 を参照してください。）

合同式の考え方では例えば 10 を法として、

- $5 + 6 \equiv 1 \pmod{10}$ であるが、 $5 + 6 \neq 1$ である。
- 整数は無限個ある。 a, b に対して $x \equiv a + b \pmod{10}$ となる x も無限個ある。
- 整数は 10 を法として 10 種類に分類できる。 a, b に対して $x \equiv a + b \pmod{10}$ となる x 全体は、そのうちちょうど 1 種類分である。
- 1, 11, 21, 31, ... 等は異なる数であるが、10 を法とする加算・減算・乗算においては同じように扱える。

ということになります。一方で環 $\mathbb{Z}/10\mathbb{Z}$ では、合同式において同じように扱える数をより明示的にひとつの数としてまとめて扱っており、

- $5 + 6 = 1$ である。
- 数（環の要素）はちょうど 10 個ある。
- a, b に対して $x = a + b$ となる x もちょうど 1 個である。

ということになります。合同式で「異なる数であるが同じように扱える」という考え方をしていたものを、単にひとつの数として扱えるようになり、議論が簡潔になることがあります。

6. modint クラス

計算結果を $\text{mod } m$ で求めることが目的である場合、「加算・減算・乗算のたびに m による剰余に置き換える」という操作が頻発します。このような操作を自動で行うような実装がある程度一般的に用いられています。

例えば AtCoder Library では、`static\_modint` と `dynamic\_modint` という 2 つのクラスが実装されています。

- 実装：<https://github.com/atcoder/ac-library/blob/master/atcoder/modint.hpp>

- ドキュメント：https://atcoder.github.io/atcoder-library/production/document_ja/modint.html

法 m がプログラムの実行前（コンパイル時）に与えられている場合には `static_modint`，そうでない場合には `dynamic_modint` を利用することが想定されています。これは m が実行前に与えられている方が，プログラムが剰余の計算をより高速に行えるためです。

これまでの内容を踏まえると，`static_modint` に実装されているメソッドのうち `pow`，`inv` 以外のものについては実装を何となく理解できると思います（どちらかというと，言語仕様の理解の方が大変だと思います）。

`modint` クラスは，環 $\mathbb{Z}/m\mathbb{Z}$ の実装だとしてもよいでしょう。演算の定義された集合（代数系）の実装の入門的な題材としても適切なもので，興味のある方は実装方法を勉強してみてください。

ただし，AtCoder Library はかなり作りこまれたライブラリです。よりシンプルな実装を参考にしたい場合には，例えば [文献 5](#) などを参考にしてください。

7. 関連問題

1. https://atcoder.jp/contests/abc266/tasks/abc266_b
2. https://atcoder.jp/contests/abc006/tasks/abc006_2
3. https://atcoder.jp/contests/arc149/tasks/arc149_a
4. https://atcoder.jp/contests/typical90/tasks/typical90_x
5. https://atcoder.jp/contests/abc086/tasks/arc089_a
6. https://atcoder.jp/contests/arc113/tasks/arc113_b
7. https://atcoder.jp/contests/abc179/tasks/abc179_e
8. https://atcoder.jp/contests/abc339/tasks/abc339_f

▶ 問題について

8. まとめ

本講座では**合同式**の基礎的な考え方について説明し， $\text{mod } m$ での計算は，基本的には計算のたびに m での剰余に置き換えるという単純な方法で出来ることを確認しました。競技プログ

ラミングでは特に「答えを m で割った余りで出力せよ」という形式の問題のほとんど全てで、この考え方が用いられることになります。

正しい計算方法を理解すると同時に、【問題 5】で述べたような誤った推論を行ってしまうことがないように注意してください。

合同式に関連する整数論の基礎的な事項は AtCoder Algorithm Lectures における以下の講座でも扱っているので、順に学んでいただけるとありがたいです。

- [Euclid の互除法](#)
- [素数の性質](#)
- [素数を法とする多項式](#)
- [原始根](#)
- [中国剰余定理](#)

9. 参考文献

1. Wikipedia. 整数の合同. <https://ja.wikipedia.org/wiki/整数の合同>. (閲覧日: 2026-03-31).
2. えびちゃん. mod についてふんわりとしか理解していない競プロ er のみなさん. <https://rsk0315.hatenablog.com/entry/2023/05/05/221524>. (閲覧日: 2026-03-31).
3. けんちゃん. 「998244353 で割ったあまり」の求め方を総特集！ ～ 逆元から離散対数まで ～. <https://qiita.com/drken/items/3b4fdf0a78e7a138cd9a>. (閲覧日: 2026-03-31).
4. USACO Guide. Modular Arithmetic. <https://usaco.guide/gold/modular>. (閲覧日: 2026-03-31).
5. noshi91. modint 構造体を使ってみませんか？ (C++). <https://noshi91.hatenablog.com/entry/2019/03/31/174006>. (閲覧日: 2026-03-31).



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#) [🔗](#)

資料請求

[AtCoder](#) [🔗](#)

[AtCoderJobs](#) [🔗](#)

各サービス

[AtCoder](#) [🔗](#)

[AtCoderJobs](#) [🔗](#)

[アルゴリズム実技検定](#) [🔗](#)

[AtCoderCareerDesign](#) [🔗](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#) [🔗](#)

[プライバシーポリシー](#) [🔗](#)

[利用規約](#) [🔗](#)

